

Article

A High-Performance Branch Control Mechanism for GPGPU Based on RISC-V Architecture

Yao Cheng ¹, Yi Man ^{1,*} and Xinbing Zhou ²¹ School of Electronic Engineering, Beijing University of Posts and Telecommunications, Beijing 100876, China; chengyao@bupt.edu.cn² School of Information and Communication Engineering, Hainan University, Haikou 570228, China; xinbingzhou@hainanu.edu.cn

* Correspondence: manyi@bupt.edu.cn

Abstract

General-Purpose Graphics Processing Units (GPGPUs) rely on warp scheduling and control flow management to organize parallel thread execution, making efficient control flow mechanisms essential for modern GPGPU design. Currently, the mainstream RISC-V GPGPU Vortex adopts the Single Instruction Multiple Threads (SIMT) stack control mechanism. This approach introduces high complexity and performance overhead, becoming a major limitation for further improving control efficiency. To address this issue, this paper proposes a thread-mask-based branch control mechanism for the RISC-V architecture. The mechanism introduces explicit mask primitives at the Instruction Set Architecture (ISA) level and directly manages the active status of threads within a warp through logical operations, enabling branch execution without jumps and thus reducing the overhead of the original control flow mechanism. Unlike traditional thread mask mechanisms in GPUs, our design centers on RISC-V and realizes co-optimization at both the ISA and microarchitecture levels. The mechanism was modeled and validated on Vortex SimX. Experimental results show that, compared with the Vortex SIMT stack mechanism, the proposed approach maintains correct control semantics while reducing branch execution cycles by an average of 31% and up to 40%, providing a new approach for RISC-V GPGPU control flow optimization.

Keywords: RISC-V; GPGPU; ISA; thread mask; branch divergence

1. Introduction

With the development of heterogeneous computing, the GPGPUs have become one of the mainstream accelerator architectures, able to handle a wide range of computational tasks. The data level parallelism (DLP) of GPGPUs is usually realized based on the SIMT execution model. SIMT is a high-level abstraction at the programming model and thread scheduling level. Each thread executes independently, with its own instruction address and register state. Threads within a warp are of the same type and start from the same program address. Instructions are broadcast synchronously to the active threads in the warp, while individual threads may become inactive due to independent branching or predicates [1–3]. At the hardware level, GPU execution units fundamentally perform parallel execution in a Single Instruction Multiple Data (SIMD) model, threads are mapped to the lanes of the underlying execution units [2,4,5].

Under the SIMT execution model, GPGPUs leverage parallel programming models such as CUDA [5] and OpenCL [6] for multithreaded scheduling. This enables efficient



Academic Editor: Spyridon Nikolaidis

Received: 19 November 2025

Revised: 17 December 2025

Accepted: 25 December 2025

Published: 26 December 2025

Copyright: © 2025 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

utilization of data parallelism, significantly boosting computational speed while maintaining relatively low hardware resource overhead. This particular suitability for data-parallel computing makes GPGPUs a core acceleration platform for large scale parallel computing scenarios, widely applied in fields such as scientific computing and artificial intelligence [7]. In these applications, GPGPUs not only handle computationally intensive workloads but also act as an important computing platform with relatively low power consumption thanks to their high degree of parallelism.

Meanwhile, both academia and industry are exploring lighter and more open architectures to meet the demands for flexibility and portability across diverse application scenarios. In recent years, with the emergence of the open-source RISC-V instruction set [8–10], GPGPU designs based on the RISC-V architecture have gradually become a research hotspot, attracting widespread attention from both academia and industry. Compared to traditional closed source architectures, the RISC-V architecture offers high customizability and modular extensibility, providing a solid foundation for building GPGPU systems.

GPUs use two mechanisms to schedule the execution of multiple threads. First, threads are grouped into warps, and in each cycle, the warp scheduling mechanism selects the active warps for execution [11]. Second, the control flow management mechanism determines which threads within an active warp are active. The control flow management mechanism handles the differences in thread execution paths caused by divergences, loops, and function calls during execution. It can be controlled or assisted by software through instructions in the ISA to achieve optimal and efficient thread scheduling. In GPUs, a group of threads within a warp usually executes the same instruction in parallel. When encountering conditional statements that require different control flow paths, branch divergence occurs, and the branch control mechanism is needed to decide the execution paths [12].

Due to the occurrence of divergence, the execution paths are split into multiple sub-paths, resulting in serialized execution and performance degradation. Therefore, GPGPUs still face challenges in methods involving control flow divergence decisions. The handling of thread divergence and reconvergence is essential for preserving consistent execution and efficient thread scheduling [13]. Modern GPUs rely on dedicated control flow management mechanisms to coordinate thread activity, deciding which threads in a warp should participate in the execution of each instruction. These mechanisms can be controlled or assisted by software through instructions within the ISA to achieve optimal and efficient thread scheduling [14]. All complete GPU ISAs provide standard branch instructions. NVIDIA's PTX [15] and SASS rely on predicate-based control to manage thread activation, while AMD's CDNA [16] adopts a thread mask mechanism. In addition, AMD's GCN [17] introduces explicit split and join instructions to directly manage thread divergence and reconvergence. Improving program performance in the presence of divergence has also become an active research topic in recent years. Saumya et al. employed compiler techniques, such as DARM, to reduce divergence under the SIMT execution model [18]. Bagies et al. focused on accelerating parallel execution by minimizing branch divergence [19]. P Plebański et al. mitigated execution divergence in GPGPU computing through dynamic data remapping [20].

Research on GPGPUs based on the RISC-V architecture is still in its early stage of development. Vortex developed a GPGPU system solution based on RISC-V scalar instructions and designed a minimal instruction set extension to support the SIMT execution model [21]. Ventus adopted the RISC-V vector instruction extension to construct a GPGPU architecture, exploring the application of vectorization in parallel processing [22,23]. These pioneering works fully demonstrate the flexibility and customizability of the RISC-V architecture for GPGPU design, providing a solid foundation for subsequent research.

Due to the dependence on the RISC-V, branch prediction is not supported in the GPU design. In Vortex, the SIMT stack mechanism is adopted to manage control flow. The split instruction pushes information about the current thread mask (tmask), the new thread mask and Program Counter (PC) onto the Immediate Post-Dominator (IPDOM) stack [24], while the join instruction pops this information during reconvergence [25]. To reduce the performance overhead caused by divergent branches, this paper introduces a thread-mask-based branch control mechanism for RISC-V GPGPU. It maintains thread mask information in dedicated registers and updates it through logical operations with predicate results, thereby controlling the activity of each thread and realizing control flow. This approach eliminates the need for complex branch stack and PC jump, significantly reducing the pipeline overhead caused by control path switching. At the same time, due to the characteristics of the RISC-V architecture, the proposed thread mask mechanism was adapted accordingly, making it different from traditional thread mask mechanisms. The design and execution model were implemented on the RISC-V GPGPU platform Vortex SimX [26], and the performance overhead caused by divergence was evaluated through microbenchmark testing.

This paper makes the following contributions:

- We propose a high-performance branch control mechanism under the RISC-V instruction set architecture and design a mask control instruction set based on logical operations.
- The RISC-V instruction set architecture restricts the storage of thread masks in conventional thread-local registers, so the thread mask is stored in Control and Status Registers (CSRs) instead, which differs from traditional GPU architectures such as AMDs. The proposed branch control mechanism was adapted to this architectural characteristic.
- In Vortex, the original SIMT pipeline relies on the PC and IPDOM stack to manage control flow, which cannot support the thread mask mechanism. To address this limitation, the proposed mechanism was modeled and validated on the Vortex SimX platform.

The rest of this paper is structured as follows. Section 2 reviews prior research related to control flow and briefly introduces the control flow characteristics of mainstream instruction set architectures. Section 3 provides a comprehensive explanation of the proposed control mechanism, outlines the existing SIMT stack mechanism in the RISC-V architecture, and presents a comparative analysis between the two. Section 4 describes in detail the modeling and implementation of the mechanism, while Section 5 presents the performance evaluation and discussion of the experimental results. Conclusions are provided in Section 6.

2. Analysis of Related Works

2.1. GPGPU Branch Control Mechanisms

Designing an efficient control flow management mechanism is crucial and challenging for modern GPUs. To address this, researchers have developed various software and hardware approaches to manage control flow. Early work by Wilson W.L. Fung et al. explored efficient branch mechanisms on SIMD-based GPUs, achieving a 20% performance improvement under branch scenarios [24]. The NVIDIA Tesla architecture later laid the foundation for the SIMT parallel computing model [2,27]. Minsoo Rhu et al. introduced a dual-path execution model by modifying the GPU hardware, which achieved an average performance improvement of 14.9% compared to single-path execution [28]. Sana Damani et al. proposed a Subwarp interleaving approach that leveraged thread divergence characteristics to effectively hide pipeline stalls during divergent execution phases when warp occupancy is

low [29]. However, most of these studies were based on NVIDIA architecture and used NVIDIA's simulation tools, GPGPU-Sim [30] and its enhanced version, Accel-Sim [31], for modeling and validation.

Currently, the control flow mechanisms used in mainstream GPU ISAs are shown in Table 1. In the NVIDIA architecture, control flow relies on the branch predicate mechanism. The main idea is to attach predicate registers at the instruction level to indicate whether each thread participates in the execution of the current instruction. The hardware uses an implicit stack to manage and reconverge divergent paths. Control flow is implicitly managed in hardware, but the implicit stack introduces some hardware complexity. In contrast, AMD's GCN and CDNA architectures adopt the branch thread mask model, which explicitly records the active status of each thread through the EXEC mask register to achieve thread level conditional execution and reconvergence control. Mask operations allow precise control of each thread's active state but require explicit updates, increasing the management complexity at both the compiler and ISA levels. GCN keeps a minimal stack but primarily depends on thread-mask-based control. The later RDNA [32] architecture follows the same direction and reduces the latency of mask updates by using a more compact execution unit design. The GEM [33] architecture, in contrast, integrates both mechanisms into a single framework. At the hardware layer, GEM introduces Sub-path Flag (SPF) registers to manage active thread paths and combines split/join control mechanism to achieve hierarchical branch management. This approach integrates the flexibility of mask-based control with the structured characteristics of stack-based flow control, aiming to balance flexibility and performance in complex branch structures. But in this mechanism, precise synchronization is required between the SPF management logic and the split/join scheduling logic, and the compiler must be adapted for coordinated optimization of both control mechanisms. In contrast, the Vortex architecture follows a model closer to split/join, maintaining branch contexts and reconvergence points through a hardware stack, which introduces additional latency and overhead [21,25].

Table 1. The control flow mechanisms used in mainstream GPU ISAs.

ISA	Control Flow
PTX	branch predicate
GCN	thread mask and split/join
CDNA	thread mask
RDNA	thread mask
GEM	SPF regs and split/join
Vortex	split/join

2.2. RISC-V ISA

The RISC-V ISA is specified by RISC-V International. The base instruction set in the RISC-V specification provides a set of general-purpose instructions and register files, supporting basic integer operations, load/store, and control functions. To increase flexibility, RISC-V reserves opcode space and CSR addresses while offering standard extensions, allowing adaptation to different processor implementations [8,9]. Therefore, all RISC-V processors must support the base instruction set and may optionally implement additional extension instructions and CSR registers according to their design.

The base instruction set includes 32 general-purpose registers (GPRs) that store operands and results needed during program execution, serving as the main data storage units. Register x0 is hardwired to 0, while the other registers can be used for general computation. CSRs store processor control information and status and can be read or written using special CSR instructions, providing an interface for the operating system

and higher-level scheduling logic [9]. The extensible instructions and CSRs offer flexible support for implementing control mechanisms in GPGPUs.

2.3. RISC-V GPGPU Research Based on Vortex

Research based on Vortex covers a complete path from ISA extensions, microarchitecture modeling, and software toolchain integration to application development. The project is still evolving and has become an important platform for open GPGPU research. Early work on Vortex defined the ISA extensions required to support the SIMT execution model on RISC-V [21,25]. These efforts allow Vortex to remain compatible with the RISC-V compilation ecosystem while supporting the OpenCL parallel programming model [26]. Vortex also implements a scalable multi-core GPGPU on FPGA and provides a cycle-accurate internal simulator named SimX, with a cycle accuracy within 6% of the actual Verilog model [25,26,34,35]. Figure 1 shows the microarchitecture of Vortex and its hierarchical organization. The host program running on the CPU offloads workloads to Vortex. Vortex adopts a multi-core architecture composed of multiple cores, where each core schedules multiple warps internally and dispatches workloads to the corresponding execution units.

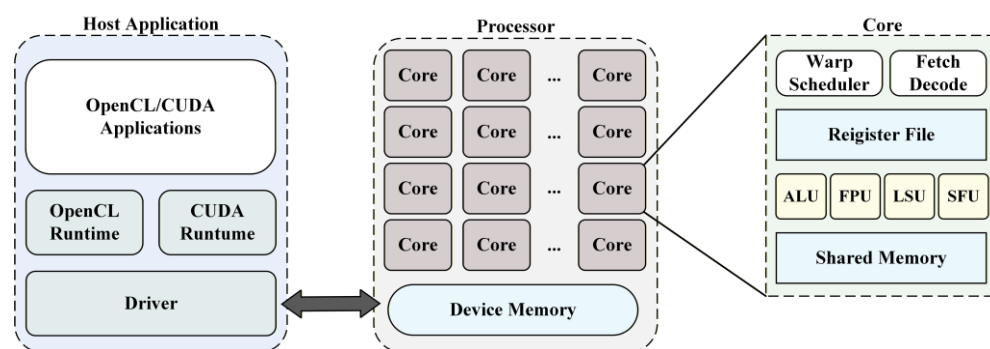


Figure 1. Vortex microarchitecture [25].

Skybox proposed a microarchitecture to accelerate Vulkan graphics Application Programming Interface (API) [36]. Chihyo Ahn et al. examined how HLS and the soft GPU Vortex compiled GPU languages for FPGA [37]. To improve ecosystem compatibility, several works [35,38,39] ported CUDA workloads to Vortex. Martin Troiber et al. explored the benefits of vectorization for Vulkan applications on a RISC-V GPU by adding vector extensions in SimX [34]. These studies showed that research on RISC-V GPUs was growing rapidly based on Vortex. However, its control mechanism still relies on the traditional SIMT stack, and the optimization space has not been fully explored. Our work aims to improve this limitation.

3. Branch Control Method

When threads within the same warp take different control paths upon encountering a conditional branch, the architecture must provide a mechanism to coordinate their divergence and reconvergence. In Section 3.1, we describe the original SIMT stack mechanism in the current Vortex architecture, and the following sections provide a complete introduction to our thread-mask-based control mechanism. The two mechanisms reflect different control flow management models in terms of execution semantics and architectural design.

3.1. SIMT Stack Mechanism

When divergence occurs within a warp, the system generates multiple sub-paths based on the branch predicates of individual threads, where each path corresponds to a specific subset of threads. Two new instructions, split and join, are introduced to manage

thread divergence and reconvergence in parallel execution [25]. During branch execution, information about inactive branches must be preserved, which is handled through an SIMT stack mechanism. In the Vortex architecture, this functionality is implemented using an IPDOM scheme, and each warp is equipped with its own stack to maintain correct coordination and restoration of thread execution paths [21,26].

In a reconvergence mechanism that uses a post-dominator stack [24], each entry corresponds to a specific branch level. When a warp encounters a branch instruction, the SIMT stack module determines the next instruction address according to the branch condition, ensuring that every thread in the warp follows the correct execution path. At the same time, the module tracks the branch condition for each thread to determine whether it satisfies the condition and selects the appropriate execution path accordingly.

By inserting a split instruction before the branch and a join instruction after it, the SIMT stack mechanism can be implemented. The detailed divergent execution process is shown in Algorithm 1 (summarized based on Vortex [25]).

Algorithm 1 Branch Execution with SIMT Stack

Input: warp PC, active thread mask active_mask, branch predicate cond, next PC next_pc

Output: updated warp PC and thread mask

```

1:  then_mask = active_mask  $\wedge$  cond
2:  else_mask = active_mask  $\wedge$   $\sim$ cond
3:  push (active_mask, else_mask, next_pc) onto IPDOM Stack
4:  next_mask = mask with more active threads
5:  set next_mask to warp active mask
6:  execute threads in next_mask
7:  retrieve current thread stack pointer stack_ptr (saved IPDOM Stack depth)
8:  if stack_ptr  $\neq$  current IPDOM Stack size then
9:      if IPDOM Stack is empty then
10:         error: stack underflow
11:      end if
12:      if top of stack fallthrough flag is true then
13:         active_mask = top.orig_mask
14:         pop top of stack
15:      else
16:         active_mask = top.else_mask
17:         PC = top.PC
18:         mark top.fallthrough = true
19:      end if
20:  end if

```

When entering a branch, the split instruction divides the control paths and pushes branch point information onto the stack. Each stack entry stores the target PC of the new branch, the reconvergence PC, and the thread mask corresponding to the threads diverging to that branch. The join instruction triggers a stack pop at the reconvergence point, restoring the active state of threads and merging the control paths, ultimately completing the reconvergence of the warp and concluding branch execution [21,25].

As shown in Figure 2, the program starts from the entry node and reaches node A, where it branches. Depending on the condition, it follows different execution paths B and C. Both B and C have D as their post-dominator, and threads reconverge at node D. The arrows in the figure indicate the active threads in the warp at each state (taking a

four-thread example). The control mechanism inserts a split before entering the branch. The split pushes the contexts of each path onto stack entries, as shown in Table 2, and pops the entries when entering the corresponding path. Table 2 shows the entry information inside the IPDOM stack in the SIMT stack mechanism.

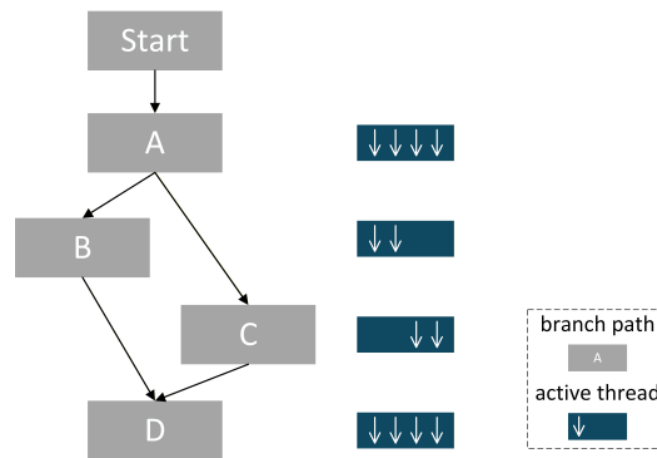


Figure 2. Thread active status.

Table 2. IPDOM entries during branch execution.

Ret./Reconv.PC	Next PC	Active Mask
-	D	1111
D	C	0011
D	B	1100

3.2. Thread Mask Mechanism

3.2.1. Overview

The fundamental idea of the thread mask control mechanism is to manage thread activity directly through a dedicated mask register, effectively transforming dynamic control flow into logical operations on the thread mask. The system assigns each warp a private thread mask register, where each thread is represented by a single bit activity flag. These flags are uniformly encoded as bitmasks and stored within the thread mask register. A bit value set to 1 indicates that the corresponding thread is active in the current cycle and participates in subsequent instruction scheduling and execution. Conversely, a bit value set to 0 means that the thread is masked out during that cycle. By performing bitwise operations on the mask, the architecture enables explicit control over thread-level execution flow.

When a divergence occurs within a warp, the system first computes a dynamic Boolean mask based on the branch predicates. The thread mask register is then updated using logical operation instructions such as AND, OR, and XOR, explicitly controlling the execution paths of individual threads and adapting the thread mask to the specific branch condition. The updates to the thread mask rely entirely on these dedicated mask logic instructions. The results of these operations are used solely to update the state of the thread mask register and do not consume resources from the conventional Arithmetic Logic Unit (ALU). Unlike conventional branch mechanisms that rely on jumps, this mechanism advances branches under a single PC path. To support reversible thread states and path merging, the system stores the current thread mask before branch divergence. After all sub-paths are executed, a mask restore instruction is used to roll back the thread mask, achieving thread reconvergence without jumps or stack operations, as shown in Algorithm 2.

Algorithm 2 Branch Execution with Thread Mask**Input:** current thread mask `exec_mask`, branch condition `cond`**Output:** updated thread mask `exec_mask`

```

1:  exec_start = exec_mask
2:  exec_if = exec_start  $\wedge$  cond
3:  for each thread i in warp do
4:      if exec_if[i] == 1 then
5:          execute_if_path (thread i)
6:      end if
7:  end for
8:  exec_else = exec_start  $\oplus$  exec_if
9:  for each thread i in warp do
10:     if exec_else[i] == 1 then
11:         execute_else_path (thread i)
12:     end if
13: end for
14: exec_mask = exec_if  $\vee$  exec_else

```

The branch execution process is shown in Figure 3. When the branch condition is satisfied, the set of truth value threads generated by the Boolean predicate will be mask-activated and enter the if branch path, threads that do not satisfy the condition will activate in the else branch. After evaluating the conditional predicate, a Boolean flag is generated for each thread. The logical AND operation instruction performs a bitwise AND operation between this flag bit and the current thread mask, yielding the thread mask for the condition being true. Simultaneously, the logical AND operation instruction directly updates the thread mask register and preserves the mask state prior to divergence. The warp then proceeds to execute the if branch. Using logical XOR instruction to perform an XOR operation between the initial thread mask and the thread mask of the if branch generates the thread mask corresponding to the else branch. To execute multiple branches or other complex branching logic, the corresponding masks can be constructed by combining multiple predicate results with logical operation instructions. During the thread convergence phase, since the warp records the mask state before entering the divergence, convergence can be achieved either by restoring the initial mask directly through the thread mask set instruction, or by gradually restoring the thread states of all sub paths using the mask merge instruction, allowing all threads to reunite on a unified execution path.

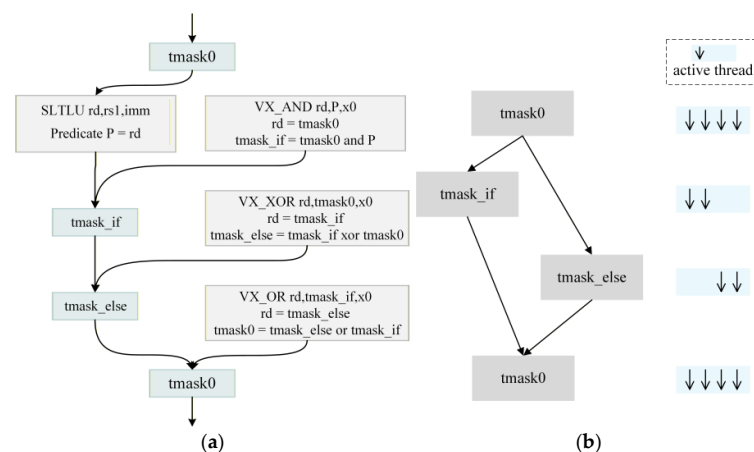


Figure 3. Control flow in the thread mask mechanism: (a) Divergence and reconvergence process; (b) Thread active status.

3.2.2. Extension of Mask Logical Operation Instruction Set

RISC-V is an open-source instruction set architecture that follows the Reduced Instruction Set Computing (RISC) philosophy. Its base instruction set employs a fixed 32-bit instruction length, defining different instruction formats through opcodes, function codes (func3, func7), and register fields (rd, rs1, rs2) to define different instruction formats. The most common formats include R-type, I-type, and S-type.

The RISC-V instruction set specification reserves custom opcode space to support user-defined extensions. The logical operation instructions extended in this study all adopt the R-type instruction format, as shown in Figure 4. R-type instructions define a 3-bit func3 field and a 7-bit func7 field. The operand combination follows the general form $C = f(A, B)$. For single input operations, one of the operands is set to the integer register x0, which always holds the value of 0.

[31:25]	[24:20]	[19:15]	[14:12]	[11:7]	[6:0]	Type
fn7	rs2	rs1	fn3	rd	op	R

Figure 4. R-type format.

To support fine-grained control flow management at the thread level, this paper proposes and implements eight mask logic operation extensions to the RISC-V instruction set. These instructions are specifically designed to perform Boolean logic transformations on the thread mask, dynamically adjusting the activation state of threads to handle complex control flow divergence and reconvergence patterns. All new instructions follow a consistent semantic structure: before performing the logical operation, the current thread mask is saved to the destination register for subsequent state recovery or mask merging. Subsequently, thread mask is updated based on the result of the logical operation between the source operand and the old mask.

The new instructions not only cover conventional Boolean operations (AND, OR, XOR) but also include their negated logical forms (such as NAND, NOR, XNOR, ANDN2, ORN2), enabling more flexible expression of active thread sets. If only using basic logical operations, additional NOT instructions or extra registers are often required to construct new target masks, which increases control overhead. By directly supporting negated logic at the instruction level, the control flow code sequence can be significantly simplified and register pressure can be reduced. The new instruction set encompasses operations as shown in Table 3.

Table 3. Control flow logic operation extension instruction set.

Instruction Name	Instruction Format	Operation Logic
vx_and	vx_and dst,src	tmask = (src & tmask);
vx_xor	vx_xor dst,src	tmask = (src ^ tmask);
vx_or	vx_or dst,src	tmask = (src tmask);
vx_and2	vx_andn2 dst,src	tmask = (src & ~tmask);
vx_orn2	vx_orn2 dst,src	tmask = (src ~tmask);
vx_nand	vx_nand dst,src	tmask = ~(src & tmask);
vx_nor	vx_nor dst,src	tmask = ~(src tmask);
vx_xnor	vx_xnor dst,src	tmask = ~(src ^ tmask);

3.2.3. CSR Register Constraint

In the implementation of mask state updates, different GPU architectures exhibit different register models. AMD GPUs employ the exec mask to indicate currently active work items, stored in special scalar general-purpose register (SGPR) within a wavefront [16].

Essentially, it is still a scalar general-purpose register, allowing any logical instruction to modify it directly, including both ordinary logical instructions and control flow logic instructions. That is, the target register for standard logical instructions can be either the exec register itself or any other general-purpose register. This design ensures flexible mask modification and preservation.

In GPUs designed based on the RISC-V architecture, the exec register is implemented as a dedicated CSR that can only be modified by control flow logic instructions. This mechanism ensures structured management of the thread mask state but reduces the direct programmability of logical instructions. This means that mechanisms like in traditional GPUs, which rely on logical instructions to directly modify the exec register, cannot be implemented. To balance flexibility and security, this study employs the Vortex TMC instruction to assist in branch reconvergence. TMC is an extension instruction in RISC-V designed for GPU adaptation, which allows the desired thread mask to be provided as input and directly updates the warp's thread mask.

In this mechanism, thread reconvergence can still be achieved through control flow logic instructions. However, when the process becomes overly complex, the TMC instruction cooperates to complete reconvergence, enabling the RISC-V architecture to realize equivalent thread-mask operations under CSR constraints. This mechanism separates mask updates from the general instruction domain, making thread-activity management more deterministic and providing a feasible approach for implementing fine-grained thread control in RISC-V GPGPUs.

3.3. Mechanism Comparison

The SIMT stack mechanism can maintain high execution efficiency in scenarios with long branch paths. In such cases, the frequency of branch instructions is relatively low, and the overhead of branching and stack operations occupies only a limited proportion of the total execution cycles; thus, it does not significantly affect the overall performance. In contrast, the thread mask mechanism switches control flow by updating masks without the need for branch instructions, effectively reducing pipeline stalls and instruction overhead, thereby achieving higher execution efficiency.

In highly divergent scenarios, the need to maintain multiple masks simultaneously and the increased number of idle threads may incur additional execution cycle overhead. Therefore, in complex branching scenarios, both mechanisms incur significant overhead. As the branch path length decreases or the branching complexity increases, the SIMT stack mechanism triggers frequent stack push/pop and PC jump operations, which substantially increase the burden on control logic and storage resources. When there are numerous short branches, the performance of the SIMT stack mechanism degrades rapidly; in contrast, under complex branch structures such as nested branches, the thread mask mechanism tends to experience greater performance loss. Furthermore, since the SIMT stack mechanism requires a hardware IPDOM stack to preserve execution context, each warp must maintain its own independent branch stack. As the degree of parallelism increases, the total stack resource consumption also rises, leading to higher hardware implementation and maintenance costs.

4. Proposed Thread Mask Control Mechanism Model

4.1. Proposed System Architecture Overview

The Vortex GPU architecture is organized into three hierarchical levels: Cluster, Socket, and Core. It employs a six-stage in-order issue and out-of-order commit pipeline [25]. Our work makes adjustments based on this structure, and the modified pipeline is shown in Figure 5. Each warp consists of a group of threads that share a single PC and follow the

same execution path. While instruction fetch, decode, and issue are shared within the same warp, each thread maintains its own set of private general-purpose scalar registers. In branch scenarios, threads within a warp may diverge across different execution paths. To ensure branch correctness, a thread mask is employed to record the execution of each thread individually, thereby guaranteeing correct branch behavior. In the Vortex GPGPU pipeline, the warp serves as the basic unit for execution scheduling. During runtime scheduling, the warp fetches instructions from the instruction cache and sends them to the decoder for parsing, waiting to be issued to the execution units. The execution units include the Arithmetic Logic Unit, Floating-Point Unit (FPU), Load/Store Unit (LSU), and Special Function Unit (SFU), each responsible for executing different types of instructions. The execution results are ultimately written back to the thread registers, completing the full instruction execution cycles [21].

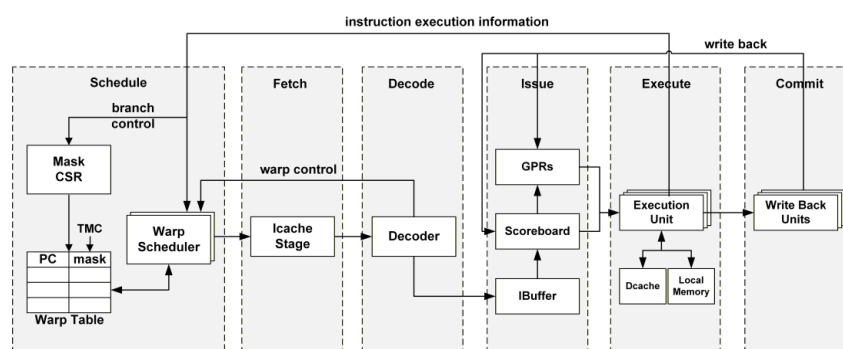


Figure 5. SIMT pipeline. (The figure is adapted from the Vortex architecture [25], with additional pipeline modifications introduced in this work).

4.2. Proposed Modeling Control Semantics and Mask Consistency

Unlike the traditional thread mask mechanism, the mechanism in RISC-V only executes conditional judgment for active threads when entering the branch, and inactive threads do not participate. This design also brings a potential correctness risk: if the inactive thread directly skips the judgment, it may lead to partial mask errors of the inactive thread of the final mask.

The adaptation scheme proposed in this paper under the RISC-V architecture adopts an explicit mask control model, and the active set of branch execution is explicitly calculated by logical instructions and merged into the current thread mask. The core idea is that each branch decision should be executed under the constraints of the current active thread collection to ensure the semantic consistency of conditional judgment and mask update. The logic is shown in Algorithm 3.

Algorithm 3 Explicit Thread Mask Update

Input: current thread mask `exec_mask`, branch condition `cond`

Output: updated thread mask `tmask`

```

1: for each thread i in warp do
2:   if exec_mask[i] == 1 then
3:     new_tmask[i] = cond[i]
4:   else
5:     new_tmask[i] = 0
6:   end if
7: end for
8: tmask = new_tmask

```

This mechanism achieves synchronized consistency between thread evaluation and mask updates by performing a bitwise merge of the condition results with the current active mask, preventing inactive threads from incorrectly affecting the mask and constraining the propagation of erroneous states. From the perspective of control semantics, this design introduces a mask protection principle in the SIMT execution model, whereby the active status of all control statements must be constrained by the existing mask to maintain consistency in parallel execution.

4.3. Proposed Module Design

4.3.1. Embedded Assembly Wrapping

To address the lack of thread mask operation support in the standard RISC-V instruction set, this study employs inline assembly to wrap custom thread mask instructions, enabling direct invocation of thread mask logic operations and flexible read and write access to the thread mask register at the software level. This approach allows low-level mask operations to be embedded directly within high-level language code, as illustrated in Figure 6. The program source code is first compiled by the Clang frontend compiler into standard intermediate representation (IR). Using an encoding-embedding approach, the LLVM-Vortex backend compiler is enhanced to recognize the instructions and compile the code onto RISC-V extended instructions, statically linking the Vortex library functions to produce an executable binary. This binary is then loaded into Vortex, where it is decoded, issued, and executed through the Vortex pipeline, with the results ultimately written back to the registers and influencing the control flow.

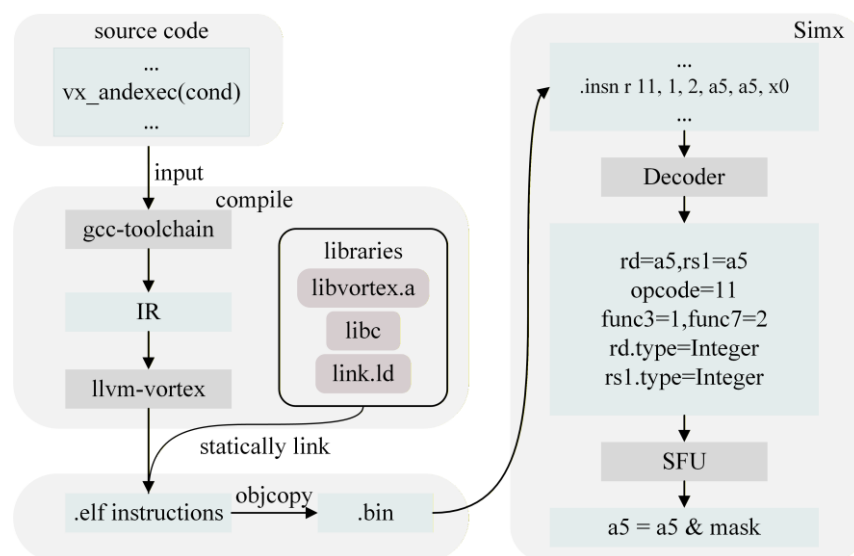


Figure 6. Program auto-generation and execution.

The implementation details are as follows:

1. Custom instruction encoding design: By using the “.insn” pseudo-instruction mechanism in RISC-V, the system dynamically combines the opcode, funct3, funct7 fields, and operand registers to implement full encoding for thread mask logic instructions, including AND, OR, XOR, and their inverse operations.
2. Dynamic machine code generation: For different thread mask operations, the system can automatically generate the corresponding machine code, bypassing compiler restrictions on unrecognized instructions, thus enabling efficient instruction insertion.
3. High-level language interface encapsulation: We used GCC/C++ version 11 as the C++ compiler in our experiments. C++ function interfaces are provided to encapsu-

late complex low-level mask logic operations into easy-to-use APIs, enabling rapid invocation and testing at the software level.

4.3.2. Thread Mask Register Read-Write Mechanism

The simulation system employs a dedicated thread mask register to store the active status of threads within each warp, enabling fine-grained management of thread execution paths. The register is integrated into the simulation architecture as a CSR, each with a unique CSR address for identification. The compiler maps high-level language operations to CSR read and write instructions through register renaming and inline assembly. During the instruction scheduling stage, the simulator parses the CSR address and executes the corresponding operation, achieving a complete simulation workflow for thread mask management.

At the simulator, CSR read instructions trigger the access logic of the corresponding CSR, retrieving the current thread mask value from the specified CSR address for instruction stream execution. CSR write instructions store the value from a general-purpose register into the specified CSR, updating the thread mask register of the corresponding warp and modifying the active thread set. In addition, the control flow logic instruction set can be used to directly modify the thread mask during instruction execution, enabling conditional execution control. In branch-switching scenarios, general-purpose registers work in conjunction with thread mask registers to manage the execution context, implementing a mechanism for saving and restoring thread mask states, thereby ensuring that thread mask management remains consistent with SIMT execution semantics.

4.3.3. Decoder

The instruction decoder, as the core component controlling execution paths, is responsible for decoding instructions. To enable correct recognition of the extended instruction set, the Vortex decoder has been functionally enhanced in this work. During the decoding stage, the system first parses the binary instruction into an instruction object usable by subsequent execution modules, extracting the opcode field from the RISC-V instruction encoding to distinguish between standard and extended instructions, and further identifying them using the func7 and func3 fields. After instruction matching, the decoder maps the required register types for thread mask operations to the target instruction object, thereby enabling an automated mapping from binary instruction codes to executable instruction objects.

4.3.4. Instruction Functional Simulation and Execution Flow

To support thread activity management under different conditions, this study introduces a set of logic operation instructions targeting the thread mask registers. The system can flexibly adjust the active thread set at the instruction level while preserving the execution context prior to switching, facilitating subsequent reconvergence or nested control.

This study integrates the thread mask control instruction set into the execution section of the SFU and designs a unified execution flow for it. Along a single PC path, the dynamic update of the thread mask registers is used to implement conditional execution and masking for different threads. The specific execution flow follows a unified instruction processing framework, comprising four key steps:

1. Decoding and instruction identification. Locate the corresponding operation instruction.
2. Context saving. During instruction execution, traverse all threads within the warp to obtain their active status and save the current mask state into the destination integer register, creating a snapshot of the state.

3. Logical operation. For each thread, perform bitwise operations on the source operand and the original mask according to the type of logical operation (e.g., AND, OR, XOR, and their inverses) to generate a new thread mask.
4. State write-back. Update the thread mask with the computed result, directly modifying the thread mask register value, thereby dynamically adjusting the active thread set within the warp.

The core idea of the thread mask mechanism is that divergent paths are directly determined by the mask, meaning that once the active threads are identified, the execution path is defined. The PC always advances sequentially, and instructions from all branch paths are executed in order. As a result, the essence of control flow changes in the proposed mechanism shifts from PC jumps to mask updates, where the CSR registers serve as the carrier of the mask state.

From a microarchitectural perspective, the proposed mask logic instructions only update the thread mask during the execution stage and participate in determining thread activity, while inactive threads are masked out. This mechanism does not introduce new pipeline stages and does not modify the instruction issue or commit process. Updates to the thread mask affect the active thread information stored in the warp table, which in turn influences warp scheduling and thereby changes the control flow within a cycle.

4.3.5. SIMT Pipeline Optimization

In the issue stage of the pipeline, the simulation system realizes the instruction allocation by round-robin scheduling. The pipeline will set the transmission bandwidth according to the size of `ISSUE_WIDTH` in each cycle, and traverse multiple adjustable warps in turn, and select candidate instructions from its instruction buffer to be sent to the execution unit. During this process, the system monitors data dependencies and resource usage through a scoreboard mechanism, ensuring that instructions only enter the execution pipeline when no conflicts exist. When execution stalls are caused by control flow, the pipeline records these events through the performance statistics module. Once instructions enter the issue stage, this module incorporates the accumulated stall cycles into the global cycle count, thereby ensuring precise modeling of the overall pipeline timing.

4.4. Hardware Implications and Feasibility Analysis

During the integration of the thread mask mechanism into Vortex, each pipeline stage was appropriately extended to support the new instructions, while the original pipeline structure and stage division remained unchanged. In the decoder, support for the new instruction binary encodings was added. In the execution stage, the thread mask within a warp is updated through bitwise operations, which are mapped to the SFU in the pipeline, but fundamentally still perform integer logic computations with the same complexity as standard RISC-V integer logic instructions. The original mask state is written back to the register after execution, involving only a small-scale register write with latency comparable to a normal register write. Mask updates occur during the execution stage of control instructions and subsequently influence scheduling decisions through the warp table, with the scheduler directly reading the updated thread mask from the table. Therefore, the mechanism has minimal impact on the critical path delay, which remains dominated by computation and memory operations [40].

In terms of area overhead, the thread mask mechanism mainly introduces a small amount of logic resources related to thread masks, including mask logic units and lightweight extensions to existing CSRs. The SIMT stack mechanism requires additional IPDOM stack structure and more stack entries, as well as push/pop logic and reconvergence management for the stack [25]. In contrast, the thread mask mechanism only needs

to maintain thread masks at the warp level and a small amount of path state in the GPRs. The mask logic consists of simple bitwise Boolean operations; its hardware implementation scales linearly with the warp width. Overall, the area overhead is controllable and remains lower than the hardware resources needed to maintain a dedicated IPDOM stack.

The thread mask mechanism does not primarily aim to reduce the number of control flow instructions. In fact, the number of logic instructions may be higher than in instruction sequences using split/join, the two mechanisms differ fundamentally in their sources of power consumption. Each instruction under the thread mask mechanism involves a very limited hardware activity, since it mainly consists of bitmask flips and a small number of register operations, rather than activating large-scale control structures, such as stack operations or reconvergence logic for post-dominator points. From an architectural perspective, this mechanism shifts power overhead from complex hardware control structures to simple logic operations.

5. Results and Discussion

5.1. Experimental Design

To systematically evaluate the performance of the two GPGPU branch control mechanisms under different parallelism configurations, this chapter conducts microbenchmark testing based on the Vortex cycle-level simulator SimX. Since variations in parallelism affect resource utilization, the underlying hardware simulation resources, such as memory, are kept fixed during the experiments. By adjusting the number of threads, warps, and cores within the GPU, multiple parallelism configurations are implemented. This setup allows for analyzing how different branch control mechanisms impact GPU execution efficiency when divergences occur within warps under various parallelism levels.

The microbenchmarks in this work are designed with reference to the existing GPGPU benchmark suite Rodinia [41]. Since the compiler development is still in progress, we constructed C++ GPU kernel test microbenchmarks that implement control and execution logic close to the assembly level. The benchmark consists of two modules. In Section 5.2, we present the first module. In this module, the microbenchmark selects Rodinia OpenCL examples. We only downgraded the kernel functions written in OpenCL and the host functions based on the OpenCL API to C++ versions that can be directly adapted to the LLVM-Vortex backend and the Vortex driver. In Section 5.3, we present the second module. Due to the large data size and computational intensity of the OpenCL examples, overall performance is easily affected by memory and arithmetic operations, making it difficult to accurately assess the performance of the branch control mechanism. To enable a more precise performance comparison, we extracted typical branch structures from the Rodinia OpenCL examples (bfs, gaussian, nearn, psum, transpose) and adapted them for testing on Vortex. The selected examples cover common branch patterns and can effectively reflect the performance characteristics of the branch control mechanisms.

5.2. Overall Performance Evaluation

To accommodate Ventus architecture, we set the parallelism to warp = 8 and thread = 32. Figure 7 shows the execution of several Rodinia examples on Vortex and Ventus. Since Ventus adopts vector ISA and its Cooperative Thread Array (CTA) scheduler differs from Vortex's warp generation strategy, the dedicated warp scheduling and dual-issue logic can significantly improve parallelism, resulting in outstanding performance for Ventus [22]. Under scalar ISA GPGPU architectures, Vortex using the thread mask control mechanism still outperforms using the SIMT stack mechanism. The performance advantage is less pronounced for some test cases because the arithmetic and memory operation overhead in the kernels is much higher than the control flow overhead.

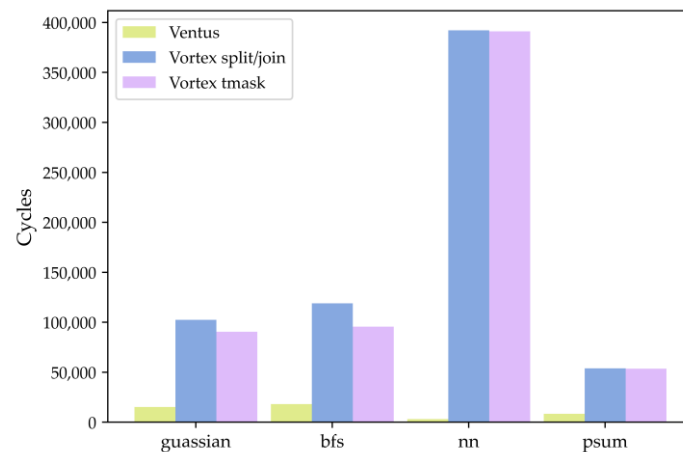


Figure 7. Comparison of cases for the three mechanisms.

5.3. Branch Control Performance Evaluation

Since each kernel may contain multiple warps and threads, all experimental results in this section have been processed to avoid duplicate counting. The figures only show performance data for the overhead of divergent instructions on a per-thread basis. In the following, “core” refers to the number of cores per socket, “warp” refers to the number of warps per core, and “thread” refers to the number of threads per warp, rather than the total number within the entire GPU.

The x-axis labels in this section are used to provide an overview of different types of branch structures, which were all extracted from Rodinia benchmark examples and are described in Section 5.1, covering typical divergence patterns in GPGPU application scenarios. Low-Degree and High-Degree represent low-complexity branch structures without further nested branches, where “low” and “high” indicate the degree of divergence: “low” corresponds to simple cases with only two branches, while “high” corresponds to multi-branch with higher divergence. Data-Dependent indicates that the branch conditions are not fixed and depend on dynamically computed predicates within the threads. Asymmetric and Multi-Level both represent high-complexity branch structures, where the thread paths within a warp are divided layer by layer, including both asymmetric and symmetric paths. In our microbenchmarks, the nesting depth is set to the most frequently occurring two-level nesting in the benchmarks. In the Rodinia test cases, each case containing branch structures can be mapped to one or more of the above scenarios. Therefore, these x-axis labels can systematically reflect the behavior of the control mechanisms under different branch patterns, enabling a comprehensive evaluation of performance.

5.3.1. Impact of Parallelism

The execution cycles of the two mechanisms under different degrees of parallelism are shown in Figure 8.

Figure 8a compares the actual overhead of the two branch control mechanisms on Vortex under the conditions of Core = 1 and Warp = 4. It can be observed that in a single core SIMT architecture with limited resources, branch overhead increases as thread-level parallelism rises, due to resource conflicts and increased branch complexity. When thread divergence occurs within a warp, the thread mask mechanism shows an advantage under low divergence. For complex branches with frequent switching, the performance of both mechanisms degrades to varying degrees.

Figure 8b shows the execution cycles of the two branch control strategies on the GPU under the configuration of Core = 1 and Thread = 16 with different numbers of warps. Increasing the number of warps mainly serves to hide long-latency operations, such as

cache misses, by improving thread-level and memory-level parallelism [21]. In this test, because the cache hit rate is high, the change in execution cycles caused by increasing the number of warps is not significant.

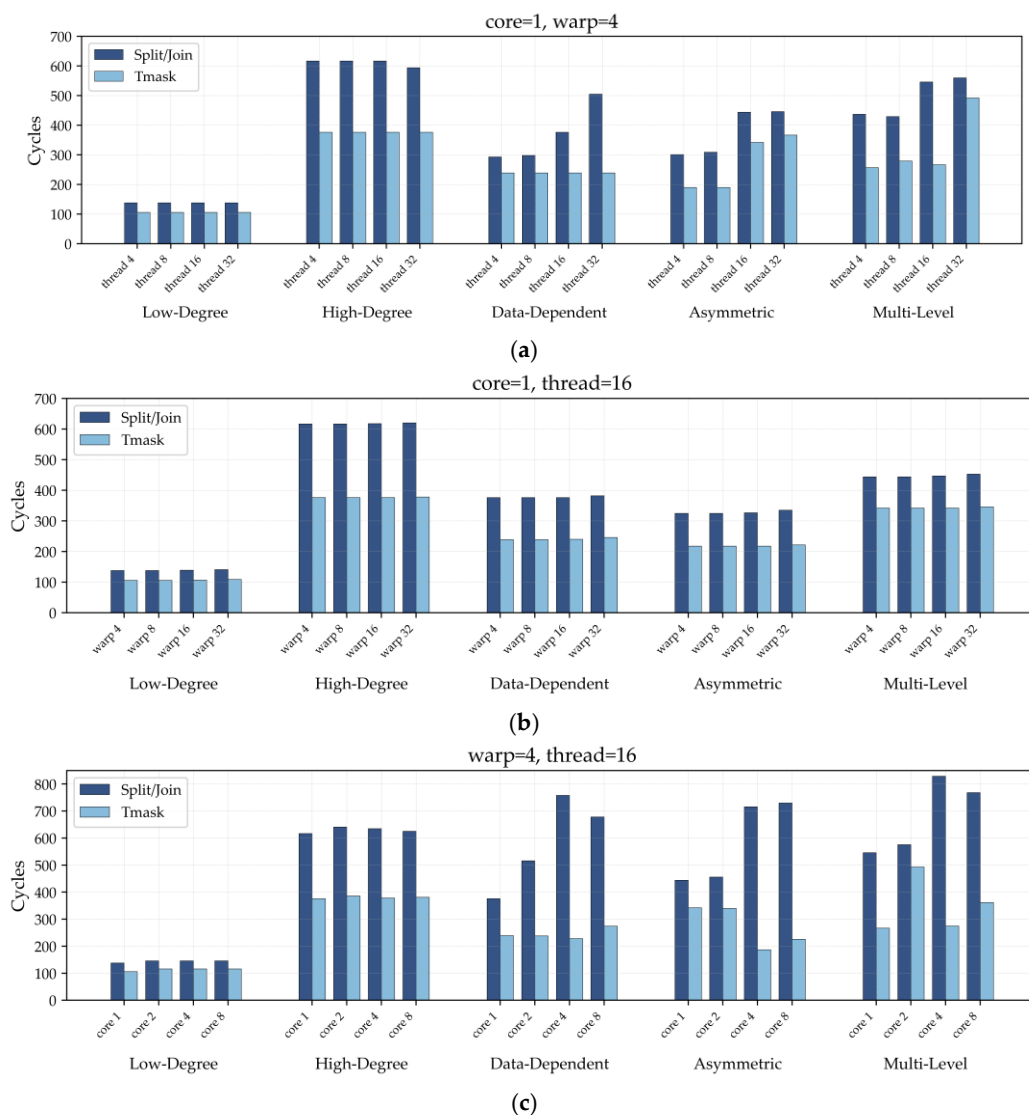


Figure 8. Execution cycles of the two mechanisms under different degrees of parallelism: (a) Core = 1, Warp = 4; (b) Core = 1, Thread = 16; (c) Warp = 4, Thread = 16.

Figure 8c shows the execution cycles of the two branch control strategies on the GPU under the configuration of Warp = 4 and Thread = 16 with different numbers of cores. As the number of cores increases, workload imbalance may occur due to GPU task partitioning, scheduling, and resource contention, causing some core tasks to be heavier and affecting thread execution cycles. Meanwhile, with fixed resources, the branch stack mechanism requires each warp on every core to maintain its own branch stack independently. As parallelism increases, the cost of context switching and stack maintenance is amplified.

When nested branches appear in the workload, the division of control paths among threads within a warp becomes more complex, and changes in parallelism can lead to significant variations in the distribution of active threads across different paths. The thread mask mechanism requires more frequent mask updates and involves multiple registers to preserve the context state, while the logic operations during reconvergence become more complex. An increased proportion of idle threads also has a considerable impact on this mechanism, causing performance fluctuations during GPU execution. For the

SIMT stack mechanism, stack operations become more frequent, and nested branches involve more reconvergence events, especially at higher levels of parallelism, resulting in an increased cost of maintaining the stack. This multi-level partitioning introduces higher control overhead, resulting in significant fluctuations in execution cycles across different parallel configurations.

Based on the experimental data above, when executing divergent branches, the thread mask control mechanism reduces branch execution cycles by an average of 31% compared to the SIMT stack mechanism, with a maximum reduction of up to 40%.

5.3.2. Impact of Divergence Level

We can observe the comparison of execution cycles for two branch control mechanisms under different levels of divergence in Figure 9. Different levels of divergence refer to the number of divergent execution paths, which are represented on the x-axis as the concrete number of paths. By executing kernels with varying numbers of branch paths on the GPU, we evaluate the impact of divergence levels on execution performance. We configured a fixed parallelism setting (core = 1, warp = 4, thread = 16) and constrained the divergence patterns within a single warp, thereby avoiding inter-warp interactions caused by data dependencies and ensuring that the control flow overhead is mainly determined by the branch control mechanism itself. Meanwhile, we ensured identical initial thread mask states (with all threads active) and consistent memory access behavior, only adjusted the control flow structure. This design guarantees that the observed performance variations are primarily caused by differences in divergence degree, with minimal influence from execution resources. Ultimately, this setup reflects the distribution differences in active threads across different execution paths within the same warp. Different divergence levels are realized by controlling the number of branch paths and the proportion of active threads in each branch relative to the total number of threads, enabling a systematic evaluation of the behavior of the two branch control mechanisms under different levels of control flow divergence.

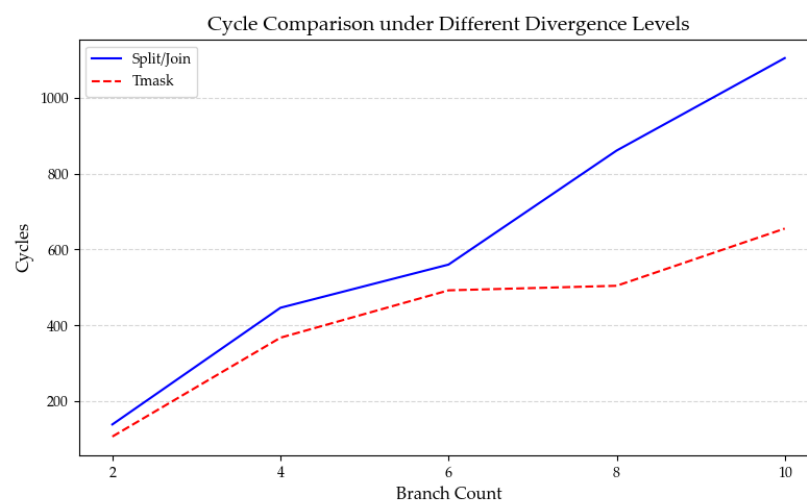


Figure 9. Execution cycles of the two mechanisms under different divergence levels.

Under low divergency, both mechanisms can be executed efficiently. Among them, the cycle of the thread mask mechanism is slightly lower than that of the SIMT stack mechanism. The thread mask mechanism maintains a performance advantage of about 18%, because it can realize conditional execution control without branch stack operations.

With increasing divergence, the execution cycles of the SIMT stack mechanism increase significantly. This is mainly due to the fact that the IPDOM stack needs to carry out frequent

stack push and pop operations when switching multiple paths, thus increasing the control overhead; and the thread mask mechanism only selects the thread through mask update, avoiding explicit jump operations, so the cycle growth is relatively slow.

In highly divergent scenarios, the execution cycles of the SIMT stack mechanism increase significantly, reflecting the continuous increase in the overhead of stack management and control. The thread mask mechanism also exhibits cycle growth, due to the increase in the number of idle threads, some computing resources are not fully utilized, which will still bring certain performance losses. At this time, the thread mask mechanism can reduce the clock overhead by about 41% compared with the SIMT stack.

5.3.3. Pipeline Performance

The pipeline performance of the two mechanisms is shown in Figure 10. The Scoreboard Stall reflects the number of stalled cycles in the pipeline caused by data dependencies or scheduling conflicts. The thread mask mechanism consistently outperforms the SIMT stack in this metric, with the average stall cycles reduced by approximately 30%, and up to 61% in the best case. The Scheduler Idle indicates the duration during which the cores remain idle, reflecting thread scheduling efficiency. The thread mask mechanism may experience some idle cycles under highly nested and unbalanced branch conditions, but it still reduces the average idle cycles by about 21% in other cases.

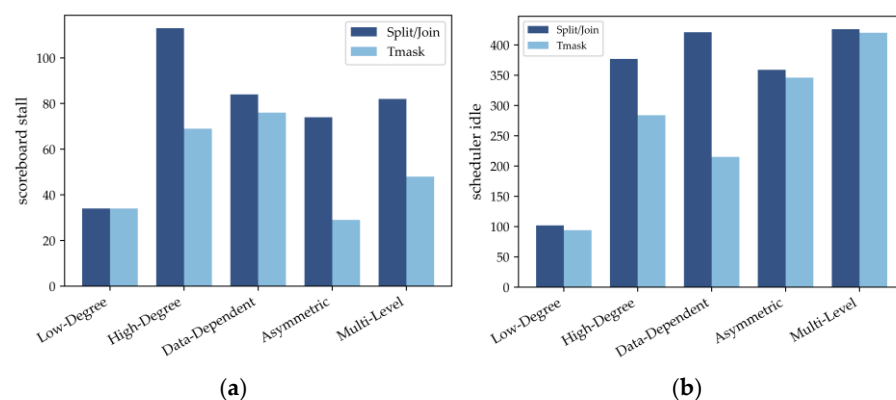


Figure 10. The pipeline performance of the two mechanisms: (a) The scoreboard stall performance; (b) The scheduler idle performance.

6. Conclusions

The study introduces and implements a high-performance branch control mechanism based on a thread mask to address control flow processing challenges in RISC-V GPGPU architectures. The original SIMT stack mechanism relies on split/join branch target instructions for control and path management, where the stack module computes target addresses for different paths, records them in stack entries, and selects the correct execution path through conditional jumps and push/pop operations. In contrast, the proposed thread mask mechanism determines thread activity through mask control instructions that apply logical operations between the thread mask and the predicate of each condition. It sequentially generates the thread masks for different paths to achieve fine grained divergence and reconvergence within a warp, thereby eliminating the need for PC jumps and branch stack operations and improving the performance of branch divergence handling. A software implementation and microbenchmark evaluation were conducted on the Vortex cycle-accurate simulator to assess the execution performance of two control mechanisms under various levels of parallelism. Experimental results show that, across diverse branch scenarios, the thread mask mechanism reduces branch instruction execution cycles by an average of approximately 31%, with a maximum reduction of 40%, while improving SIMT

pipeline performance. This work provides a feasible and efficient solution for control flow optimization in RISC-V GPGPU architectures.

This work focuses on architectural exploration and mechanism evaluation using the cycle-accurate SimX simulator, and on integrating the thread mask mechanism into the Vortex pipeline architecture. The C++ compiler is already compatible with this work, and we are currently optimizing the OpenCL toolchain to support high level parallel programming language. Future research will focus on a complete hardware implementation, further validating the advantages of the thread mask control mechanism through hardware analysis.

Author Contributions: Conceptualization, Y.C.; methodology, Y.C.; validation, Y.C., Y.M. and X.Z.; formal analysis, Y.C.; investigation, X.Z.; resources, Y.M. and X.Z.; data curation, Y.C.; writing—original draft preparation, Y.C.; writing—review and editing, Y.M.; supervision, Y.M.; project administration, Y.M.; funding acquisition, Y.M. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the National Key R&D Program of China (No.2022YFB2901100), the National Natural Science Foundation of China (No. U24A20234) and (No. 62171062).

Data Availability Statement: Data are contained within the article.

Acknowledgments: During the preparation of this work the authors used ChatGPT-5 mini in order to translate text into English. After using this tool, the authors reviewed and edited the content as needed and took full responsibility for the content of the publication.

Conflicts of Interest: The authors declare no conflicts of interest. The funders had no role in the design of the study; in the collection, analyses, or interpretation of data; in the writing of the manuscript; or in the decision to publish the results.

Abbreviations

The following abbreviations are used in this manuscript:

GPGPU	General-Purpose Graphics Processing Unit
RISC-V	Reduced Instruction Set Computer—Five
SIMT	Single Instruction Multiple Threads
SIMD	Single Instruction Multiple Data
ISA	Instruction Set Architecture
PC	Program Counter
IPDOM	Immediate Post-Dominator
CSR	Control and Status Registers
GPR	General-Purpose Registers
ALU	Arithmetic Logic Unit
FPU	Floating-Point Unit
LSU	Load/Store Unit
SFU	Special Function Unit
OPCODE	Operation CODE
TMASK	Thread Mask
SRC	Source Register
DST	Destination Register
API	Application Programming Interface
IR	Intermediate Representation
CTA	Cooperative Thread Array
CUDA	Compute Unified Device Architecture

References

1. Hennessy, J.L.; Patterson, D.A. *Computer Architecture: A Quantitative Approach*, 6th ed.; Stephenson, M., Ed.; Elsevier: Cambridge, MA, USA, 2011; p. 1525.
2. Lindholm, E.; Nickolls, J.; Oberman, S.; Montrym, J. NVIDIA Tesla: A unified graphics and computing architecture. *IEEE Micro* **2008**, *28*, 39–55. [CrossRef]
3. Nickolls, J.; Buck, I.; Garland, M.; Skadron, K. Scalable parallel programming with cuda: Is cuda the parallel programming model that application developers have been waiting for? *Queue* **2008**, *6*, 40–53. [CrossRef]
4. AMD. HIP Programming Model. Available online: https://rocm.docs.amd.com/projects/HIP/en/docs-6.2.0/understand/programming_model.html (accessed on 9 December 2025).
5. NVIDIA Corporation. CUDA C++ Programming Guide, Release 13.0. Available online: https://docs.nvidia.com/cuda/pdf/CUDA_C_Programming_Guide.pdf (accessed on 18 November 2025).
6. Munshi, A. The opencl specification. In Proceedings of the 2009 IEEE Hot Chips 21 Symposium (HCS), Stanford, CA, USA, 23–25 August 2009; pp. 1–314.
7. Dally, W.J.; Keckler, S.W.; Kirk, D.B. Evolution of the graphics processing unit (GPU). *IEEE Micro* **2021**, *41*, 42–51. [CrossRef]
8. Asanović, K.; Patterson, D.A. *Instruction Sets Should Be Free: The Case FOR RISC-V*; UCB/EECS-2014-146; EECS Department, University of California at Berkley: Berkeley, CA, USA, 2014.
9. RISC-V International. The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA. Available online: <https://riscv.org/wp-content/uploads/2019/12/riscv-spec-20191213.pdf> (accessed on 18 November 2025).
10. RISC-V International. RISC-V Official Website. Available online: <https://riscv.org/> (accessed on 18 November 2025).
11. Rogers, T.G.; O'Connor, M.; Aamodt, T.M. Cache-conscious wavefront scheduling. In Proceedings of the 2012 45th Annual IEEE/ACM International Symposium on Microarchitecture, Vancouver, BC, Canada, 1–5 December 2012; pp. 72–83.
12. Han, T.D.; Abdelrahman, T.S. Reducing branch divergence in GPU programs. In Proceedings of the Fourth Workshop on General Purpose Processing on Graphics Processing Units, Newport Beach, CA, USA, 5 March 2011; pp. 1–8.
13. Aamodt, T.M.; Fung, W.W.L.; Rogers, T.G.; Martonosi, M. *General-Purpose Graphics Processor Architectures*, 1st ed.; Martonosi, M., Ed.; Morgan & Claypool Publishers: San Rafael, CA, USA, 2018; p. 122.
14. Shoushtary, M.A.; Murgadas, J.T.; Gonzalez, A. Control Flow Management in Modern GPUs. *arXiv* **2024**, arXiv:2407.02944. [CrossRef]
15. NVIDIA Corporation. Parallel Thread Execution Instruction Set Architecture, Version 8.5. Available online: https://docs.nvidia.com/cuda/pdf/ptx_isa_8.5.pdf (accessed on 18 November 2025).
16. AMD. Amd Instinct Mi300 Cdna3 Instruction Set Architecture. Available online: <https://www.amd.com/content/dam/amd/en/documents/instinct-tech-docs/instruction-set-architectures/amd-instinct-mi300-cdna3-instruction-set-architecture.pdf> (accessed on 18 November 2025).
17. AMD. Gcn3 Instruction Set Architecture. Available online: <https://www.amd.com/content/dam/amd/en/documents/radeon-tech-docs/instruction-set-architectures/gcn3-instruction-set-architecture.pdf> (accessed on 18 November 2025).
18. Saumya, C.; Sundararajah, K.; Kulkarni, M. DARM: Control-flow melding for SIMT thread divergence reduction. In Proceedings of the 2022 IEEE/ACM International Symposium on Code Generation and Optimization (CGO), Seoul, Republic of Korea, 2–6 April 2022; pp. 1–13.
19. Bagies, T.; Le, W.; Sheaffer, J.; Jannesari, A. Reducing branch divergence to speed up parallel execution of unit testing on GPUs. *J. Supercomput.* **2023**, *79*, 18340–18374. [CrossRef]
20. Plebański, P.; Kelm, A.; Kłopotek, R.A. Mitigating Execution Divergence in GPGPU Computing through Dynamic Data Remapping. In Proceedings of the 2024 IEEE 17th International Scientific Conference on Informatics (Informatics), Poprad, Slovakia, 13–15 November 2024; pp. 281–288.
21. Elsabbagh, F.; Asgari, B.; Kim, H.; Yalamanchili, S. Vortex risc-v gpgpu system: Extending the isa, synthesizing the microarchitecture, and modeling the software stack. In Proceedings of the Third Workshop on Computer Architecture Research with RISC-V (CARRV 2019), Phoenix, AZ, USA, 22 June 2019.
22. Li, J.; Yang, K.; Jin, C.; Liu, X.; Yang, Z.; Yu, F.; Shi, Y.; Ma, M.; Kong, L.; Zhou, J. Ventus: A high-performance open-source gpgpu based on risc-v and its vector extension. In Proceedings of the 2024 IEEE 42nd International Conference on Computer Design (ICCD), Milan, Italy, 18–20 November 2024; pp. 276–279.
23. Tsinghua University DSP Lab. Ventus GPGPU. Available online: <https://github.com/THU-DSP-LAB/ventus-gpgpu> (accessed on 18 November 2025).
24. Fung, W.W.; Sham, I.; Yuan, G.; Aamodt, T.M. Dynamic warp formation and scheduling for efficient GPU control flow. In Proceedings of the 40th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO 2007), Chicago, IL, USA, 1–5 December 2007; pp. 407–420.

25. Tine, B.; Yalamarthy, K.P.; Elsabbagh, F.; Hyesoon, K. Vortex: Extending the risc-v isa for gpgpu and 3d-graphics. In Proceedings of the MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture, Virtual Event, Greece, 18–22 October 2021; pp. 754–766.
26. Elsabbagh, F.; Tine, B.; Roshan, P.; Lyons, E.; Kim, E.; Shim, D.E.; Zhu, L.; Lim, S.K. Vortex: Opencl compatible risc-v gpgpu. *arXiv* **2020**, arXiv:2002.12151.
27. NVIDIA Corporation. NVIDIA Tesla V100 GPU Architecture. Available online: <https://www.nvidia.cn/content/dam/en-zz/Solutions/Data-Center/tesla-product-literature/volta-architecture-whitepaper.pdf> (accessed on 18 November 2025).
28. Rhu, M.; Erez, M. The dual-path execution model for efficient GPU control flow. In Proceedings of the 2013 IEEE 19th International Symposium on High Performance Computer Architecture (HPCA), Shenzhen, China, 23–27 February 2013; pp. 591–602.
29. Damani, S.; Stephenson, M.; Rangan, R.; Johnson, D.; Kulkarni, R.; Keckler, S.W. Gpu subwarp interleaving. In Proceedings of the 2022 IEEE International Symposium on High-Performance Computer Architecture (HPCA), Seoul, Republic of Korea, 2–6 April 2022; pp. 1184–1197.
30. Bakhoda, A.; Yuan, G.L.; Fung, W.W.; Wong, H.; Aamodt, T.M. Analyzing CUDA workloads using a detailed GPU simulator. In Proceedings of the 2009 IEEE International Symposium on Performance Analysis of Systems and Software, Boston, MA, USA, 26–28 April 2009; pp. 163–174.
31. Khairy, M.; Shen, Z.; Aamodt, T.M.; Rogers, T.G. Accel-sim: An extensible simulation framework for validated gpu modeling. In Proceedings of the 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA), Valencia, Spain, 30 May–3 June 2020; pp. 473–486.
32. AMD. RDNA Instruction Set Architecture. Available online: https://www.amd.com/content/dam/amd/en/documents/radeon-tech-docs/instruction-set-architectures/rdna-shader-instruction-set-architecture.pdf?utm_source=chatgpt.com (accessed on 18 November 2025).
33. Intel. Intel Graphics Hardware Specifications. Available online: <https://www.intel.com/content/www/us/en/developer/topic-technology/open/overview.html> (accessed on 18 November 2025).
34. Troiber, M.; Schulz, M.; Tine, B.; Kim, H. Analysis of the RISC-V Vector Extension for Vulkan Graphics Kernels. In Proceedings of the 2025 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS), Ghent, Belgium, 11–13 May 2025; pp. 388–389.
35. Ahn, C.; Han, R.; Subramanya, U.; Zhao, J.; Tine, B.; Kim, H. SoftCUDA: Running CUDA on Softcore GPU. In Proceedings of the 2025 IEEE 33rd Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM), Fayetteville, AR, USA, 4–7 May 2025; pp. 138–142.
36. Tine, B.; Saxena, V.; Srivatsan, S.; Simpson, J.R.; Alzamar, F.; Cooper, L.; Kim, H. Skybox: Open-source graphic rendering on programmable risc-v gpus. In Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Vancouver, BC, Canada, 25–29 March 2023; Volume 3, pp. 616–630.
37. Ahn, C.; Jeong, S.; Cooper, L.P.; Parnenzini, N.; Kim, H. Comparative Analysis of Executing GPU Applications on FPGA: HLS vs. Soft GPU Approaches. In Proceedings of the 2024 IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), San Francisco, CA, USA, 27–31 May 2024; pp. 634–641.
38. Han, R.; Tine, B.; Lee, J.; Sim, J.; Kim, H. Supporting CUDA for an extended RISC-V GPU architecture. *arXiv* **2021**, arXiv:2109.00673. [[CrossRef](#)]
39. Han, R.; Chen, J.; Garg, B.; Young, J.; Sim, J.; Kim, H. CuPBoP: CUDA for Parallelized and Broad-range Processors. *arXiv* **2022**, arXiv:2206.07896.
40. Bridges, R.A.; Imam, N.; Mintz, T.M. Understanding GPU power: A survey of profiling, modeling, and simulation methods. *ACM Computing Surveys (CSUR)* **2016**, *49*, 41. [[CrossRef](#)]
41. Che, S.; Boyer, M.; Meng, J.; Tarjan, D.; Sheaffer, J.W.; Lee, S.-H.; Skadron, K. Rodinia: A benchmark suite for heterogeneous computing. In Proceedings of the 2009 IEEE International Symposium on Workload Characterization (IISWC), Austin, TX, USA, 4–6 October 2009; pp. 44–54.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.